Отчёт по финальному проекту

19 апреля 2026 г.

1 Кто чем в нашей команде занимался

- Васильева Анастасия изучила архитектуру ResNet, модифицировала её и в итоге получила финальный результат.
- Винник Маргарита эксперементировала с вейвлет-преобразованиями, преобразованием Фурье, CNN, пытаясь собрать гибридную модель.
- Кабанов Сергей изучал архитектуру F3-Net и эксперементировал с ней.

2 Постановка задачи

Задача: Написать модель распознавания дипфейков. Модель представляет собой свёрточную нейронную сеть бинарной классификации, которая будет отличать настоящие лица от сгенерированных.
В условии задачи нужно было обратить внимание на следующие нюансы:

1. Прежде всего важно, минимизировать ошибку второго рода (когда это deep fake, а модель ошибочно определила его как реального человека);
2. Переводя на метрики задачи классификации, нужно максимизировать recall. При этом за precision также важно следить, чтобы не классифицировать все как deep fake;
3. Учитывая предыдущие два пункта качество решения будет оцениваться по метрике f1-score;
4. Также в датасете наблюдается сильный дисбаланс классов.

3 Ход работы

3.1 1й эксперимент

Сначала нашей командой было принято решение посмотреть и поискать уже существующие модели, которые хорошо справляются с подобной задачей.

Внимание привлекли модели семейства ResNet - сети с остаточными соединениями, хорошо зарекомендовавшая себя в задачах классификации изображений. Как изветстно существует четыре версии по количеству слоёв (18, 34, 50, 152). Решили, что для начало следует попробовать с 18-тью слоями, т.к. датасет небольшой.

Код первой модели:

```
class DeepfakeResNet18(nn.Module):
    def __init__(self, pretrained=True, dropout_rate=0.5):
    # Почему такой dropout?
    # Это борьба с переобучением (дипфейки часто имеют мало уникальных артефактов)
        super(DeepfakeResNet18, self).__init__()
        self.resnet = models.resnet18(pretrained=pretrained)
        num_features = self.resnet.fc.in_features
        self.resnet.fc = nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(num_features, 512),
            # позволяет выучить более сложные комбинации признаков подделки
            nn.ReLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(512, 1)
        )
    def forward(self, x):
        return self.resnet(x)
```

Здесь сначала вызывается готовая модель ResNet18, на вход которой подаётся следующие данные: 3 канала (RGB), размер изображения 224×224. На выходе получаем 1000 классов (ImageNet). Понятно что умение модели распознавать 1000 классов избыточно, ведь нам нужно всего 2. Заменяем послений слой суть которого теперь ответить на вопрос «это дипфейк? да/нет», то есть ответ 0 - реальное изображение или 1 - это сгенерированное изображение.

Новый последний слой представляет собой следующую цепочку:

Dropout $\rightarrow$ Linear(512, 512) $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Linear(512, 1)

Результаты обучение данной модели (20 эпох):

Train - Loss: 0.1053, F1: 0.9277, Recall: 0.9738, Precision: 0.8858

Val - Loss: 0.2004, F1: 0.9462, Recall: 0.9443, Precision: 0.9480

Результат более чем хороший, но модель не соответствует требованиям, т.к. это готовая модель с минимальными изменениями под заданный датасет, а нас просили написать свою. Подобное решение называется **Transfer Learning** (обучение с переносом).

Думаем и эксперементируем дальше.

3.2 2й эксперимент

Добавим в каждый residual-блок Squeeze-and-Excitation (SE) механизм — лёгкий attention по каналам признаков

Идея: сеть сама учится, какие каналы важны для данного входа, и масштабирует их соответственно. Это практически не увеличивает количество параметров, но на практике даёт заметный прирост качества (статья Hu et al., 2018).

Дополнительно:

- Двухслойный классификатор с Dropout и ReLU вместо одного линейного слоя;
- Взвешенная функция потерь BCEWithLogitsLoss для борьбы с дисбалансом классов;
- Cosine Annealing LR scheduler — постепенно снижает learning rate без резких скачков;

```python
# SE-блок (Squeeze-and-Excitation)
# добавляем channel attention к каждому residual-блоку реснет-18
# сеть сама учится взвешивать каналы: типа важные усиливаются, шумные подавляются
class SEBlock(nn.Module):
    def __init__(self, channels, reduction=16):
        super(SEBlock, self).__init__()
        #squeeze: глобальный пул -> делает вектор [B, C]
        self.squeeze = nn.AdaptiveAvgPool2d(1)
        # excitation - два линейных слоя
        self.excitation = nn.Sequential(
            nn.Linear(channels, channels // reduction, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(channels // reduction, channels, bias=False),
            nn.Sigmoid()
        )

    def forward(self, x):
        b, c, _, _ = x.size()
        # глобальный average pool по пространственным измерениям
        s = self.squeeze(x).view(b, c)
        # веса для каждого канала
        e = self.excitation(s).view(b, c, 1, 1)
        # перемасштабируем
        return x * e.expand_as(x)


# обёртка
# не будем исходный код ResNet менять, просто оборачиваем каждый BasicBlock
class SEBasicBlock(nn.Module):
    def __init__(self, basic_block, channels):
        super(SEBasicBlock, self).__init__()
        self.block = basic_block
        self.se = SEBlock(channels)
```

```python
    def forward(self, x):
        out = self.block(x)    # обычный residual forward
        out = self.se(out)     # применяем channel attention
        return out

class DeepfakeResNet18(nn.Module):
    def __init__(self, pretrained=True, dropout_rate=0.5):
        super(DeepfakeResNet18, self).__init__()
        self.resnet = models.resnet18(pretrained=pretrained)

        # внедряем тут SE-блоки в каждый residual-блок всех четырёх слоёв
        # channel counts берём из стандартной ResNet18: [64, 128, 256, 512]
        se_channels = [64, 128, 256, 512]
        for layer_idx, (layer_name, channels) in enumerate(
            zip(['layer1', 'layer2', 'layer3', 'layer4'], se_channels)
        ):
            layer = getattr(self.resnet, layer_name)
            new_blocks = nn.Sequential(*[
                SEBasicBlock(block, channels) for block in layer
            ])
            setattr(self.resnet, layer_name, new_blocks)

        num_features = self.resnet.fc.in_features   #кол-во входных признаков у последне
        self.resnet.fc = nn.Sequential(
        #тут заменяем последний слой
            nn.Dropout(dropout_rate),
            nn.Linear(num_features, 512),
            nn.ReLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(512, 1)
        )

    def forward(self, x):
        return self.resnet(x)

# чек размер выхода и количество параметров
_model_test = DeepfakeResNet18(pretrained=False)
_dummy = torch.zeros(2, 3, 256, 256)
_out = _model_test(_dummy)
total_params = sum(p.numel() for p in _model_test.parameters())
trainable_params = sum(p.numel() for p in _model_test.parameters() if p.requires_grad)
print(f"Output shape: {_out.shape}  (expected [2, 1])")
print(f"Total params: {total_params:,}")
print(f"Trainable params: {trainable_params:,}")
del _model_test, _dummy, _out
```

На 20й эпохе обучения показала следующие результаты:

Train - Loss: 0.1204, F1: 0.9168, Recall: 0.9713, Precision: 0.8680
Val - Loss: 0.1503, F1: 0.9560, Recall: 0.9553, Precision: 0.9568
LR: $1.61e-06$

Лучшее значение метрики F1 у данной модели получилось следующее 0.9576 (на 19й эпохе).

4 Выводы по получившемуся решению

Архитектура:

За основу взяли ResNet18 с предобучением на ImageNet. К стандартным residual-блокам добавили Squeeze-and-Excitation attention: после каждого блока сеть вычисляет веса значимости для каждого из 64/128/256/512 каналов и перемасштабирует feature maps. Это позволяет модели концентрироваться на артефактах StyleGAN (характерные паттерны в текстуре кожи, граничные эффекты) и игнорировать шумные каналы

Данные:

Применяли аугментации: горизонтальный flip, brightness/contrast jitter, ShiftScaleRotate, GaussNoise, случайное размытие, CoarseDropout На валидации только нормализацию делали, пытались бороться с дисбалансом классов через взвешенный BCEWithLogitsLoss

Обучение:

AdamW + CosineAnnealingLR на 20 эпох (так как этого зхватило для базовой модельки для хорошего результата) Seed зафиксирован (42) хихихи

Инференс:

Вместо простого порога 0.5 (как изначально я выбрала)используем два улучшения:

- Подбор порога по валидации - перебираем пороги от 0.30 до 0.70 с шагом 0.01 и выбираем максимизирующий F1, полезная штука для прироста качества без переобучения (сначала без него сделали и моделька переобучаться начала.
- TTA (Test Time Augmentation) - тут каждое тестовое изображение прогоняется через 4 варианта (оригинал, горизонтальный флип, $+$brightness, $-$brightness), вероятности как бы усредняются, и это снижает дисперсию предсказаний на граничных случаях.

5 Неудачные эксперименты, которые не привели к хорошему результату

Пробовали реализовать следующие идеи, но это не попало в финальный ноутбук из-за проблем с вычислительными ресурсами, нормализацией или непониманием как справиться с дисбалансом классов:

- вейвлет-преобразование признаков;

- преобразование Фурье для анализа частотных артефактов GAN;
- KNN-классификатор;
- Архитектура F3-Net;

5.1 Про архитектуру F3-net

F3-Net (Frequency in Face Forgery Network) — это нейросеть для обнаружения дипфейков, которая смотрит не на само изображение, а на его частотное представление.

Обычно модели анализируют изображение поканально в цветовом пространстве RGB. При сильном сжатии (JPEG, H.264) мелкие артефакты подделки сглаживаются и становятся незаметны для таких моделей (например, ResNet). Если перевести изображение в частотную область с помощью DCT (дискретное косинусное преобразование - стандартный метод перевода изображений в частотную область, используется в JPEG), то артефакты подделки остаются видны даже после агрессивного сжатия — они проявляются как аномально сильные сигналы на определённых частотах.

F3-Net — это двухпоточная сеть с механизмом внимания (cross-attention) для слияния признаков.

Схема архитектуры:

```
Входное изображение
        │
        ├──────────────┬──────────────┐
        ▼              ▼              ▼
      FAD            LFS          (RGB поток — опционально)
   (разложение       (локальная
    по частотам)      статистика)
        │              │
        ▼              ▼
   Xception блоки   Xception блоки
        │              │
        └──────┬───────┘
               ▼
          MixBlock (cross-attention)
               │
               ▼
          Классификация (реал/фейк)
```

FAD (Frequency-Aware Decomposition) - разрезает картинку на три частотные полосы — низкие, средние и высокие частоты .

Как работает:

- Изображение → DCT (частотная область)

- Три обучаемых фильтра вырезают три диапазона частот
- Обратное DCT → получаем три новых изображения (низкочастотное, среднечастотное, высокочастотное)
- Они склеиваются по каналам и подаются в Xception

Зачем это нужно: Артефакты подделки часто видны именно в высокочастотной компоненте.

LFS (Local Frequency Statistics) - считает статистику частот в маленьких окошках (патчах) по всему изображению .

Как работает:

- Изображение режется на перекрывающиеся патчи (например, 10×10, шаг 2)
- К каждому патчу применяется DCT
- Для каждого частотного диапазона считается средняя амплитуда

Получается карта размером с исходное изображение, но с K каналами (по числу частотных полос)

Зачем это нужно: FAD возвращает изображения обратно в пространство, а LFS работает напрямую с частотами - они дополняют друг друга.

MixBlock (Cross-Attention) - смешивает признаки из FAD и LFS.

Как работает:

- Берёт карты признаков из обоих потоков
- Считает cross-attention (внимание одного потока к другому)
- Внедряет усиленные признаки обратно в каждый поток

Зачем это нужно: Просто склеить (concat) - плохо. Cross-attention позволяет потокам учиться друг у друга, выделяя самые важные для детекции дипфейков признаки.

В качестве основы для обоих потоков используется Xception (предобученный на ImageNet), у которого берут не только последний слой, а несколько промежуточных (на 7-м и 12-м блоках ставят MixBlock).

Вот наша попытка самостоятельно написать F3-net для решения задачи бинарной классификации дипфейков:

```
class FrequencyAwareDecomposition(nn.Module):
""" Модуль Frequency-Aware Decomposition """
def __init__(self, device, in_channels, reduction=16, kernel_size=3):
  super(FrequencyAwareDecomposition, self).__init__()

  base_low = T.GaussianBlur(3)
  base_mid = nn.Sequential(T.GaussianBlur(3), T.GaussianBlur(3))
  base_high = T.GaussianBlur(3)
```

```
        learn_low = nn.Conv2d(in_channels, in_channels,
        kernel_size, padding='same', groups=in_channels, bias=False).to(device)
        learn_mid = nn.Conv2d(in_channels, in_channels,
        kernel_size, padding='same', groups=in_channels, bias=False).to(device)
        learn_high = nn.Conv2d(in_channels, in_channels,
        kernel_size, padding='same', groups=in_channels, bias=False).to(device)

        learn_low.requires_grad = True
        learn_mid.requires_grad = True
        learn_high.requires_grad = True

        self.filters = [(base_low, learn_low),
                        (base_mid, learn_mid),
                        (base_high, learn_high)]

    def forward(self, x):
        y = []
        for i in self.filters:
            y.append(torch.mul(dct.idct_2d(i[1](i[0](x))), dct.dct_2d(x)))

        return torch.cat(y, dim=1).to(device)


class LocalFrequencyStatistics(nn.Module):
    """ Модуль Local Frequency Statistics """
    def __init__(self, kernel_size=7):
        super(LocalFrequencyStatistics, self).__init__()
        self.kernel_size = kernel_size
        self.unfold = nn.Unfold(kernel_size=kernel_size, padding=kernel_size//2, stride=1)
        self.project = nn.Conv2d(3 * 4, 3, kernel_size=1)

    def forward(self, x):
        B, C, H, W = x.shape
        patches = self.unfold(x)
        patches = patches.view(B, C, self.kernel_size**2, H, W)

        mean = patches.mean(dim=2, keepdim=True)        # Среднее
        var = patches.var(dim=2, keepdim=True)          # Дисперсия
        skew = ((patches - mean)**3).mean(dim=2, keepdim=True) / (var.sqrt() + 1e-6)**3 # Ас
        kurt = ((patches - mean)**4).mean(dim=2, keepdim=True) / (var + 1e-6)**2 - 3  # Эксц

        stats = torch.cat([mean.squeeze(2), var.squeeze(2), skew.squeeze(2), kurt.squeeze(2)

        return stats
```

```python
class MixBlock(nn.Module):
    """ Модуль MixBlock """
    def __init__(self, embed_dim):
      super(MixBlock, self).__init__()
      self.cross_attn = nn.MultiheadAttention(embed_dim, 8, batch_first=True)

    def forward(self, fad, lfs):
      attn_map = F.scaled_dot_product_attention(lfs, fad, fad)
      fad_out = torch.mul(attn_map, lfs) + fad
      lfs_out = torch.mul(attn_map, fad) + lfs
      return fad_out, lfs_out


class F3Net(nn.Module):
  """ Полная архетектура F3-Net """
  def __init__(self, device, backbone='resnet18', pretrained=True):
    super(F3Net, self).__init__()

    self.fad = FrequencyAwareDecomposition(device, 3).to(device)
    self.lfs = LocalFrequencyStatistics().to(device)

    m_fad = timm.create_model(backbone, pretrained=pretrained).to(device)
    m_lfs = timm.create_model(backbone, pretrained=pretrained).to(device)

    m_fad_layers = list(m_fad.children())
    m_lfs_layers = list(m_lfs.children())

    self.block1_fad = nn.Sequential(nn.Conv2d(9, 3,
    kernel_size=3, padding=1), *m_fad_layers[:4]).to(device)
    self.block2_fad = nn.Sequential(*m_fad_layers[4:6]).to(device)
    self.block3_fad = nn.Sequential(*m_fad_layers[6:]).to(device)

    self.block1_lfs = nn.Sequential(nn.Conv2d(12, 3,
    kernel_size=3, padding=1), *m_lfs_layers[:4]).to(device)
    self.block2_lfs = nn.Sequential(*m_lfs_layers[4:6]).to(device)
    self.block3_lfs = nn.Sequential(*m_lfs_layers[6:]).to(device)

    self.mixBlock = MixBlock(64).to(device)

    self.block_pred = nn.Sequential(
        nn.Dropout(0.5),
        nn.Linear(2000, 1)
    )

  def forward(self, x):
      fad_out = self.fad(x)
```

```
        lfs_out = self.lfs(x)


        block1_fad_out = self.block1_fad(fad_out)
        block1_lfs_out = self.block1_lfs(lfs_out)
        block1_fad_out, block1_lfs_out = self.mixBlock(block1_fad_out, block1_lfs_out)

        block2_fad_out = self.block2_fad(block1_fad_out)
        block2_lfs_out = self.block2_lfs(block1_lfs_out)
        block2_fad_out, block2_lfs_out = self.mixBlock(block2_fad_out, block2_lfs_out)

        block3_fad_out = self.block3_fad(block2_fad_out)
        block3_lfs_out = self.block3_lfs(block2_lfs_out)
        y = torch.cat((block3_fad_out, block3_lfs_out), dim=1)
        y = self.block_pred(y)

        return y
```

При реализации F3-Net были предприняты попытки воспроизвести архитектуру, описанную в оригинальной статье. Однако в ходе экспериментов выявились следующие проблемы:

1. Модуль Frequency-Aware Decomposition был реализован с неверной логикой - фильтрация частот происходила после обратного DCT, что нивелировало преимущества частотного подхода.

2. Local Frequency Statistics работал в пространственной области вместо частотной, что привело к потере ключевой информации об артефактах сжатия.

3. Механизм cross-attention в MixBlock был реализован с неправильной размерностью тензоров, что делало его неработоспособным.

4. Наблюдались проблемы со стыковкой размерностей между backbone'ом и финальным классификатором из-за отсутствия адаптивного пулинга.

В связи с высокой сложностью корректной реализации всех трёх компонентов и их чувствительностью к деталям, было принято решение отказаться от F3-Net в пользу более простой и проверенной архитектуры ResNet18 с дообучением последних слоёв. Это позволило сфокусироваться на качестве данных и процессе обучения, а не на отладке сложного частотного преобразования.

F3-Net - это интересная и классная архитектура на бумаге, но довольно сложная в реализации. Малейшая ошибка в DCT/IDCT или размерностях ломает всю сеть. ResNet18 хоть и проще, но работает стабильно и даёт предсказуемый результат.

5.2 CNN для бинарной классификации + вейвлет-преобразование признаков

Был исследован гибридный подход, сочетающий вейвлет-преобразование и свёрточную нейронную сеть (WaveletCNN). Вейвлеты позволяют декомпозировать изображение на частотные компоненты с сохранением пространственной локализации - в отличие от DCT, который работает глобально.

Была реализована следующая архитектура:

- Вейвлет-преобразование (Haar, level=1) с помощью библиотеки pttw для GPU-вычислений
- Формирование 12-канального тензора (LL, LH, HL, HH для каждого RGB-канала)
- Min-max нормализация каждой компоненты
- CNN-классификатор с 4 свёрточными блоками и усиленной регуляризацией (Dropout2d, Dropout 0.6/0.4)

Вот код этого забавного Франкенштейна:

```python
def to_wavelet_features(image_tensor, wavelet='haar', level=1):
    """
    Преобразует RGB тензор в вейвлет-признаки

    Args:
        image_tensor: (C, H, W) или (B, C, H, W) в диапазоне [0, 1]
        wavelet: тип вейвлета ('haar', 'db2', 'bior1.3')
        level: уровень разложения

    Returns:
        Тензор вейвлет-признаков (12, H/2, W/2) для level=1
    """
    # Добавляем batch dimension если нужно
    if image_tensor.dim() == 3:
        image_tensor = image_tensor.unsqueeze(0)
        single = True
    else:
        single = False

    # Вейвлет-разложение
    coeffs = ptwt.wavedec2(image_tensor, wavelet, level=level, mode='reflect')

    # coeffs[0] - низкочастотное приближение (LL)
    # coeffs[1] - детали: (LH, HL, HH)
    approx = coeffs[0]  # (B, C, H/2, W/2)
```

```python
    details_h, details_v, details_d = coeffs[1]  # каждый (B, C, H/2, W/2)

    # Объединяем по каналам: 4 компонента × 3 RGB канала = 12 каналов
    wavelet_features = torch.cat([approx, details_h, details_v, details_d], dim=1)

    # Нормализация каждого канала отдельно (опционально, но помогает)
    for i in range(wavelet_features.shape[1]):
        ch_min = wavelet_features[:, i, :, :].min()
        ch_max = wavelet_features[:, i, :, :].max()
        if ch_max - ch_min > 1e-8:
            wavelet_features[:, i, :, :] =
            (wavelet_features[:, i, :, :] - ch_min) / (ch_max - ch_min)

    if single:
        wavelet_features = wavelet_features.squeeze(0)

    return wavelet_features

class WaveletCNN(nn.Module):
    """
    CNN для вейвлет-признаков с усиленной регуляризацией
    Вход: 12 каналов, размер 128x128
    """

    def __init__(self, num_classes=2):
        super().__init__()
        self.features = nn.Sequential(
            # Блок 1: 12 -> 32
            nn.Conv2d(12, 32, 3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.Dropout2d(0.1),
            nn.MaxPool2d(2),  # 128 -> 64

            # Блок 2: 32 -> 64
            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Dropout2d(0.1),
            nn.MaxPool2d(2),  # 64 -> 32

            # Блок 3: 64 -> 128
            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout2d(0.1),
```

```
        nn.MaxPool2d(2),  # 32 -> 16

        # Блок 4: 128 -> 256
        nn.Conv2d(128, 256, 3, padding=1),
        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.Dropout2d(0.1),
        nn.AdaptiveAvgPool2d(1)  # 16 -> 1
    )

    self.classifier = nn.Sequential(
        nn.Dropout(0.6),  # увеличили с 0.5 до 0.6
        nn.Linear(256, 128),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),  # добавили дополнительный dropout
        nn.Linear(128, num_classes)
    )

def forward(self, x):
    x = self.features(x)
    x = x.view(x.size(0), -1)
    x = self.classifier(x)
    return x
```

Получившиеся метрики модели после 30ти эпох обучения (да, неудовлетворительные, но в отличие от F3-net она за 3-4 часа всё-таки обучилась и обработала датасет):

Train - Loss: 0.6064, F1: 0.4176, Recall: 0.6216, Precision: 0.3144

Val - Loss: 0.5872, F1: 0.4546, Recall: 0.5184, Precision: 0.4048

В ходе экспериментов выявились следующие ограничения:

1. Использование вейвлета Haar приводило к появлению ступенчатых артефактов, которые CNN ошибочно интерпретировала как признаки подделки.

2. Один уровень разложения оказался недостаточным для захвата мелких артефактов сжатия (8×8 JPEG-блоков).

3. Min-max нормализация «уравняла» значимость низкочастотной (LL) и высокочастотной (HH) компонент, что привело к усилению шума.

4. Уменьшение пространственного разрешения в 2 раза (256→128) привело к потере микро-артефактов размером 1-2 пикселя.

5. Глубокая CNN переобучалась на высокочастотных шумах, несмотря на регуляризацию.

Потенциальные улучшения: использование гладких вейвлетов (db4, bior1.3), увеличение уровня разложения до 2-3, старт с 512×512, замена min-max на стандартизацию по среднему/std, упрощение CNN-классификатора.

Однако ввиду высокой сложности подбора гиперпараметров и скромного выигрыша в точности по сравнению с ResNet18, дальнейшая работа над WaveletCNN была прекращена.